

Documentação de Projeto

para o sistema

AutoFix — Gestão de Oficina Mecânica

Versão 1.0

Projeto de sistema elaborado pelo(s) aluno(s) Gustavo Pessoa Firmino Duarte
como parte da disciplina **Projeto de Software**.

10 de junho de 2026

Tabela de Conteúdo

1. Introdução

2. Modelos de Usuário e Requisitos

2.1. Descrição de Atores

2.2. Modelo de Casos de Uso

2.3. Diagrama de Sequência do Sistema

2.4. Contratos de Operação

3. Modelos de Projeto

- 3.1. Arquitetura
- 3.2. Diagrama de Componentes e Implantação
- 3.3. Diagrama de Classes
- 3.4. Diagramas de Sequência
- 3.5. Diagramas de Comunicação
- 3.6. Diagramas de Estados

4. Modelos de Dados

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Gustavo Pessoa Firmino Duarte	10/06/2026	Elaboração inicial do documento: modelos de usuário e requisitos, modelos de projeto e modelos de dados.	1.0

1. Introdução

Este documento agrega: 1) a elaboração e revisão de modelos de domínio e 2) modelos de projeto para o sistema **AutoFix — Gestão de Oficina Mecânica**. A referência principal para a descrição geral do problema, domínio e requisitos do sistema é o documento de especificação que descreve a visão de domínio do sistema.

O **AutoFix** é um sistema de gestão para oficinas mecânicas de pequeno e médio porte. Ele informatiza o ciclo completo de atendimento da oficina: o cliente agenda um atendimento para o seu veículo; o atendente abre uma **Ordem de Serviço (OS)**; o mecânico registra o diagnóstico e emite um orçamento com serviços e peças; o cliente aprova (ou recusa) o orçamento; o mecânico executa os serviços aprovados, registrando as peças utilizadas (com baixa automática de estoque); e, por fim, o atendente finaliza a OS recebendo o pagamento — em dinheiro ou de forma eletrônica via gateway de pagamento. Em todas as etapas relevantes o cliente é notificado por e-mail. O gerente acompanha a operação por meio de relatórios gerenciais de faturamento, produtividade e estoque.

Toda a diagramação deste documento foi produzida com **PlantUML**, e os arquivos-fonte (*.puml*) acompanham este documento na pasta *diagramas/*.

2. Modelos de Usuário e Requisitos

2.1. Descrição de Atores

Nesta subseção é apresentada descrição de cada um dos atores que interagem com o sistema.

Ator	Descrição
Cliente	Proprietário do veículo. Agenda atendimentos, acompanha o andamento da Ordem de Serviço, aprova ou recusa orçamentos e consulta o histórico de manutenções do seu veículo.
Atendente	Funcionário responsável pela recepção. Cadastra clientes e veículos, abre Ordens de Serviço, gerencia agendamentos e registra o pagamento na entrega do veículo.
Mecânico	Funcionário técnico. Realiza o diagnóstico do veículo, emite o orçamento (serviços e peças), executa os serviços aprovados e registra as peças utilizadas.
Gerente	Responsável pela administração da oficina. Mantém o catálogo de serviços e o estoque de peças e gera relatórios gerenciais (faturamento, produtividade, estoque).
Gateway de Pagamento	<i>Sistema externo</i> que autoriza transações eletrônicas (cartão de crédito, débito e PIX).
Servidor de E-mail	<i>Sistema externo</i> (SMTP) utilizado para notificar o cliente sobre orçamentos emitidos, conclusão de serviços e emissão da nota de pagamento.

2.2. Modelo de Casos de Uso

Nesta subseção é apresentado o diagrama de casos de uso do sistema. Para cada um deles, utiliza-se um ID que serve de referência no restante do documento (por exemplo, UC-01 para o Caso de Uso 01).

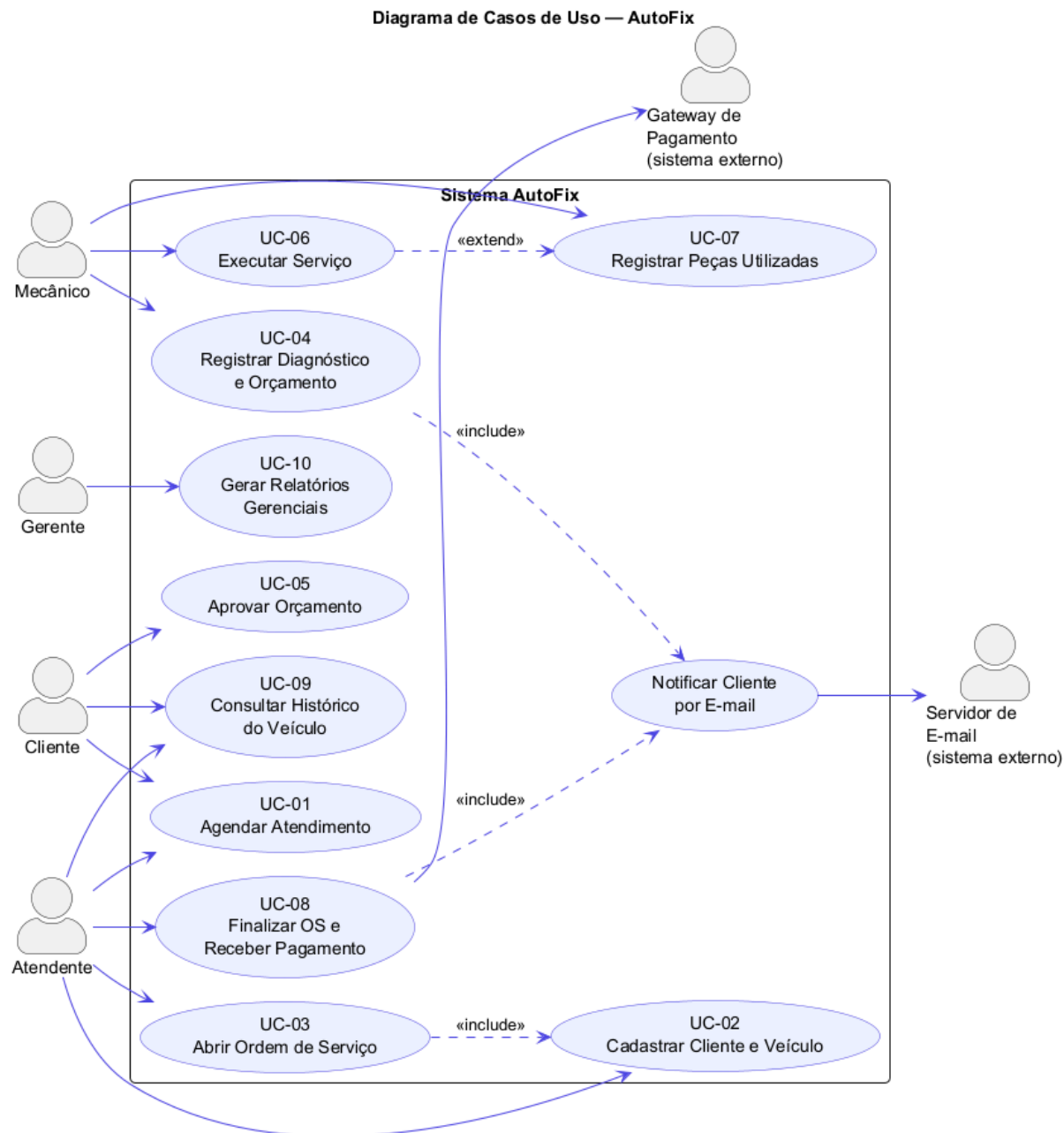


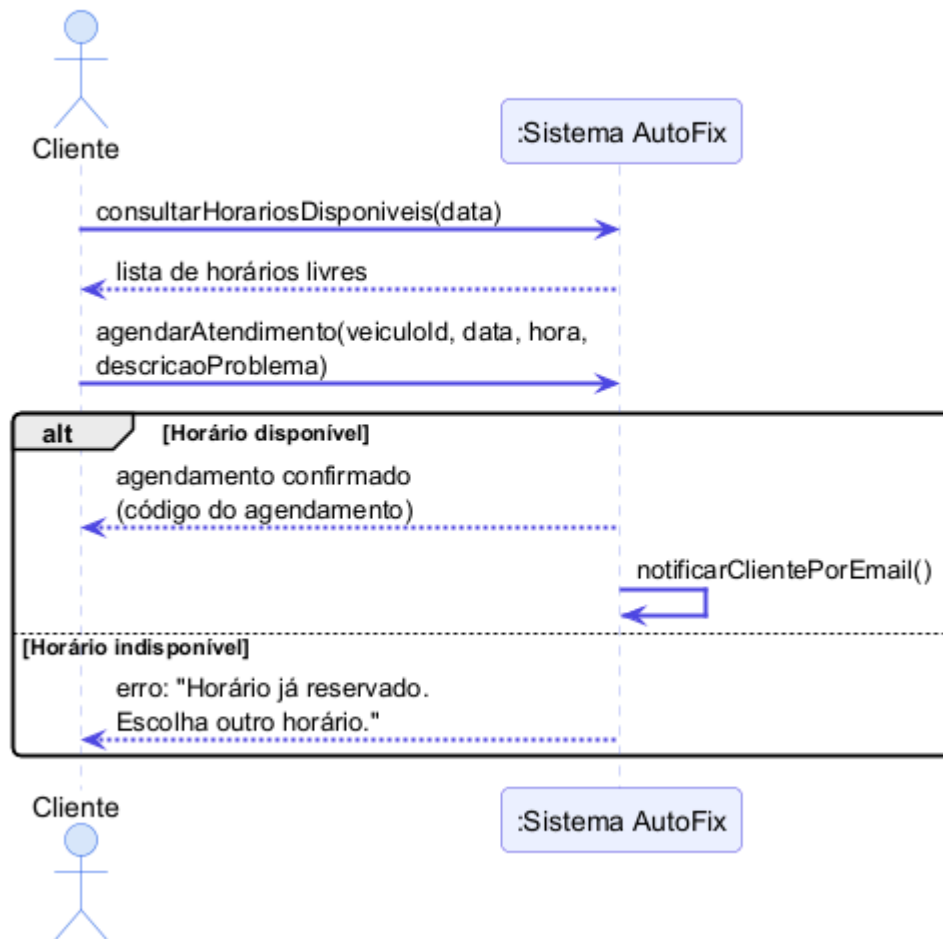
Figura 1 — Diagrama de Casos de Uso do sistema AutoFix (PlantUML).

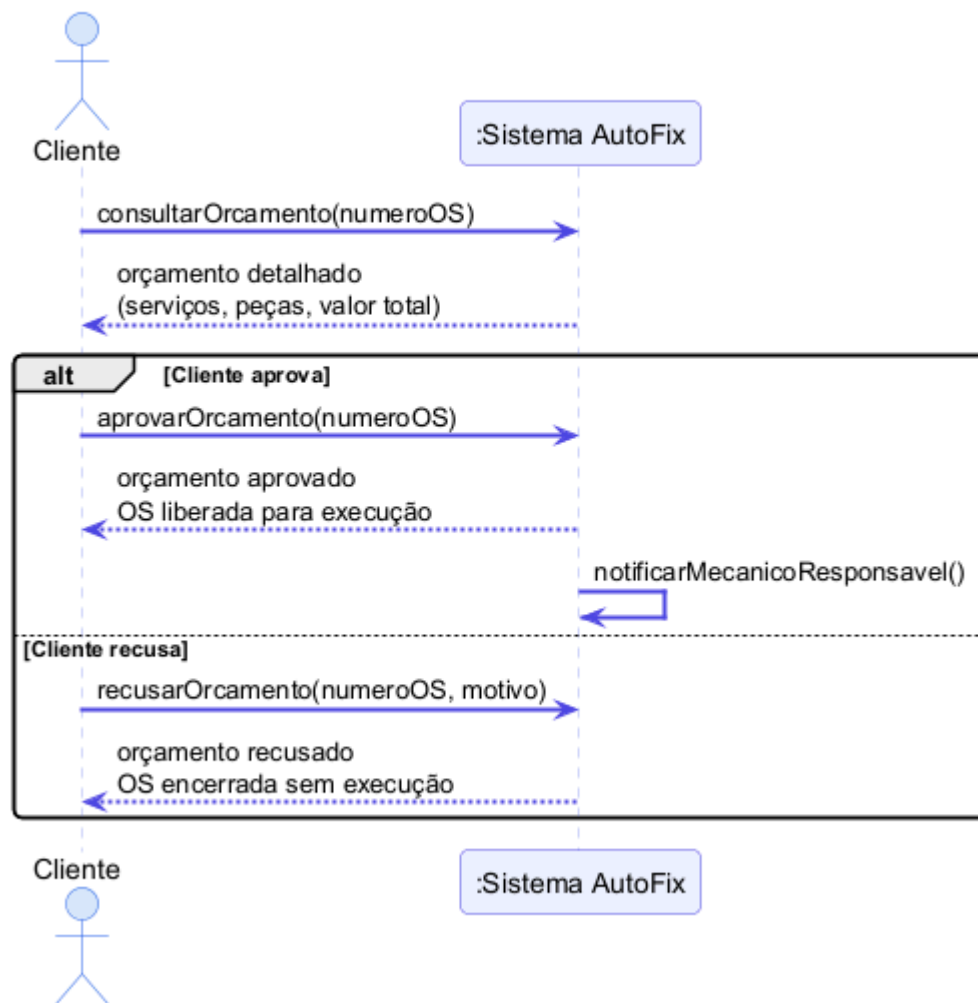
ID	Caso de Uso	Descrição resumida
UC-01	Agendar Atendimento	Cliente (ou atendente, por telefone) reserva data e hora para levar o veículo à oficina.
UC-02	Cadastrar Cliente e Veículo	Atendente registra os dados do cliente e de seus veículos.
UC-03	Abrir Ordem de Serviço	Atendente abre a OS vinculada ao veículo, com a queixa relatada pelo cliente.
UC-04	Registrar Diagnóstico e Orçamento	Mecânico registra o diagnóstico e emite orçamento com serviços e peças; o cliente é notificado por e-mail.
UC-05	Aprovar Orçamento	Cliente aprova ou recusa o orçamento emitido para a sua OS.
UC-06	Executar Serviço	Mecânico executa os itens de serviço aprovados e marca cada item como concluído.

UC-07	Registrar Peças Utilizadas	Mecânico registra as peças aplicadas na OS, com baixa automática no estoque.
UC-08	Finalizar OS e Receber Pagamento	Atendente encerra a OS e registra o pagamento (dinheiro ou eletrônico via gateway); a nota é enviada por e-mail.
UC-09	Consultar Histórico do Veículo	Cliente ou atendente consulta todas as manutenções já realizadas no veículo.
UC-10	Gerar Relatórios Gerenciais	Gerente emite relatórios de faturamento, produtividade dos mecânicos e posição de estoque.

2.3. Diagrama de Sequência do Sistema

Nesta subseção é apresentado o diagrama de sequência do sistema (DSS) de três Casos de Uso descritos na Seção 2.2: **UC-01 — Agendar Atendimento**, **UC-05 — Aprovar Orçamento** e **UC-08 — Finalizar OS e Receber Pagamento**. Em cada DSS o sistema é tratado como uma caixa-preta, evidenciando apenas os eventos trocados entre os atores e o sistema.

DSS — UC-01 Agendar Atendimento*Figura 2 — DSS do UC-01 Agendar Atendimento.*

DSS — UC-05 Aprovar Orçamento*Figura 3 — DSS do UC-05 Aprovar Orçamento.*

DSS — UC-08 Finalizar OS e Receber Pagamento

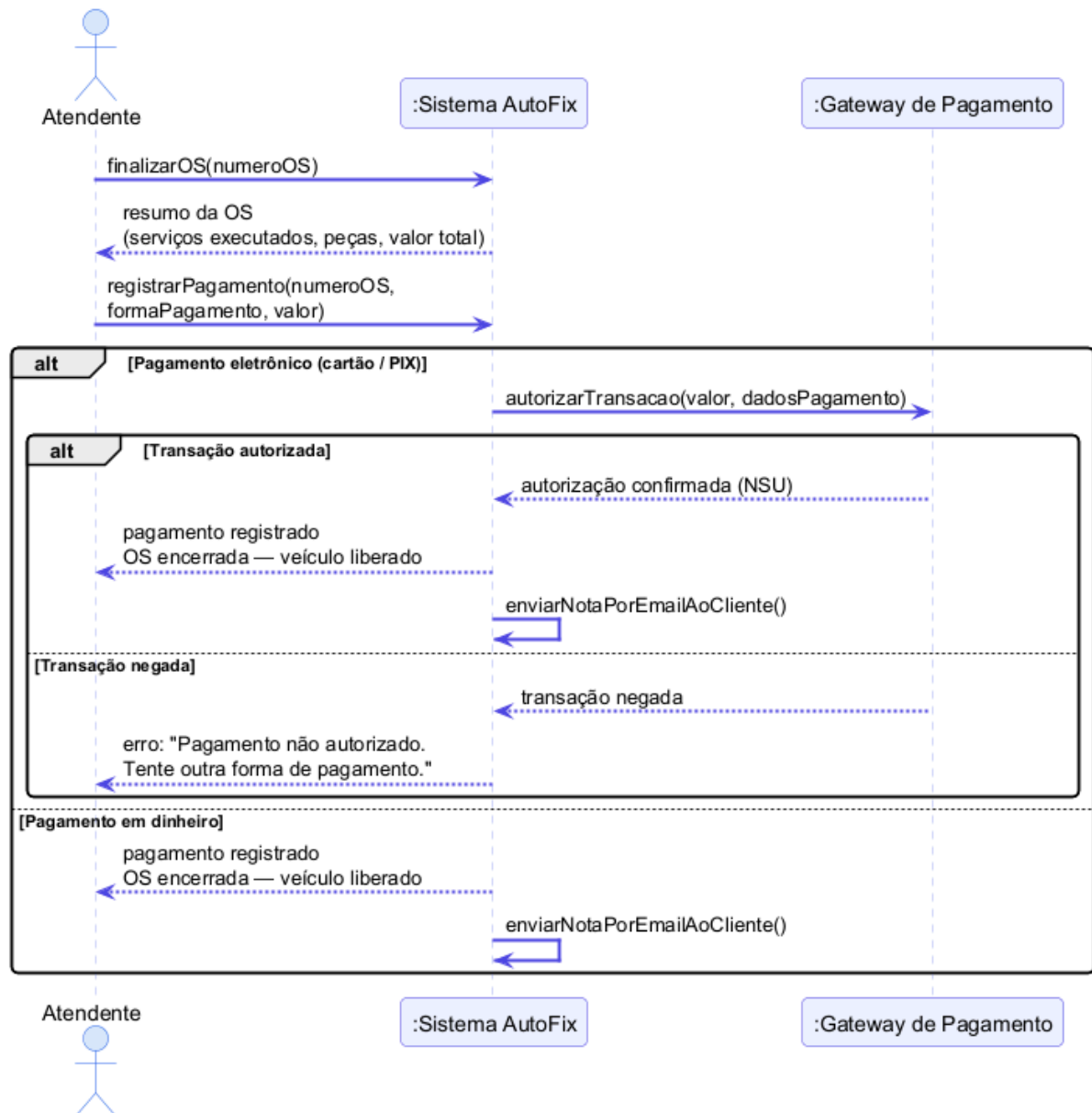


Figura 4 — DSS do UC-08 Finalizar OS e Receber Pagamento.

2.4. Contratos de Operação

Formato para cada contrato de operação:

Contrato	CO-01 — agendarAtendimento
Operação	agendarAtendimento(veiculoId, dataHora, descricaoProblema)
Referências cruzadas	UC-01 — Agendar Atendimento
Pré-condições	O cliente e o veículo estão cadastrados no sistema; o horário informado está dentro do expediente da oficina e disponível.

Pós-condições	Uma instância de <i>Agendamento</i> foi criada com status AGENDADO; o agendamento foi associado ao cliente e ao veículo; o cliente foi notificado por e-mail com o código do agendamento.
----------------------	---

Contrato	CO-02 — aprovarOrcamento
Operação	aprovarOrcamento(numeroOS)
Referências cruzadas	UC-05 — Aprovar Orçamento
Pré-condições	A OS existe e possui orçamento emitido com status AGUARDANDO_APROVACAO; o solicitante é o cliente proprietário do veículo da OS.
Pós-condições	O atributo <i>aprovado</i> do <i>Orcamento</i> foi definido como verdadeiro; o status da <i>OrdemServico</i> foi alterado para APROVADA; o mecânico responsável foi notificado para iniciar a execução.

Contrato	CO-03 — registrarPagamento
Operação	registrarPagamento(numeroOS, formaPagamento, valor)
Referências cruzadas	UC-08 — Finalizar OS e Receber Pagamento
Pré-condições	A OS está com status FINALIZADA; o valor informado corresponde ao valor total calculado da OS; em pagamento eletrônico, o gateway autorizou a transação.
Pós-condições	Uma instância de <i>Pagamento</i> foi criada e associada à OS (com NSU, quando eletrônico); o status da <i>OrdemServico</i> foi alterado para PAGA; a nota foi enviada por e-mail ao cliente; o veículo foi liberado para entrega.

3. Modelos de Projeto

3.1. Arquitetura

A arquitetura do AutoFix é descrita com um diagrama apropriado da UML. Adotou-se o estilo **arquitetura em camadas (MVC)**: uma SPA em React consome uma API REST construída em Spring Boot, organizada em camadas de apresentação (*controllers*), negócio (*services*), persistência (*repositories*) e domínio (entidades JPA), sobre um banco PostgreSQL. As integrações externas (gateway de pagamento e servidor SMTP) são acessadas exclusivamente pela camada de negócio.

Arquitetura em Camadas (MVC) — AutoFix

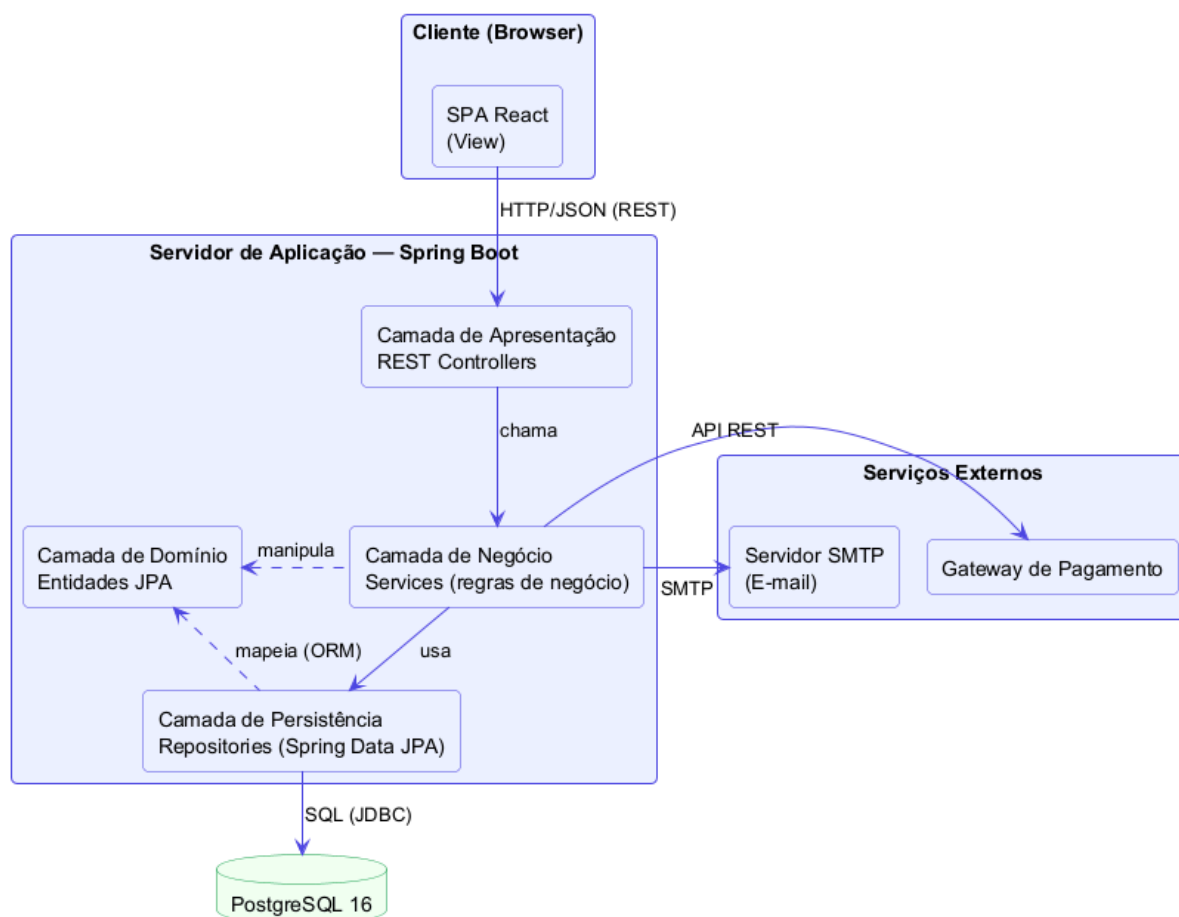


Figura 5 — Arquitetura em camadas do sistema AutoFix.

3.2. Diagrama de Componentes e Implantação

Diagramas de componentes do sistema e diagrama de implantação mostrando onde os componentes estarão alocados para a execução. O back-end é modularizado por área de negócio (agendamento, OS, orçamento, pagamento, estoque, notificação e relatórios); em produção, a aplicação roda em contêineres Docker (Nginx como proxy reverso, API Spring Boot e PostgreSQL) hospedados em um servidor cloud.

Diagrama de Componentes — AutoFix

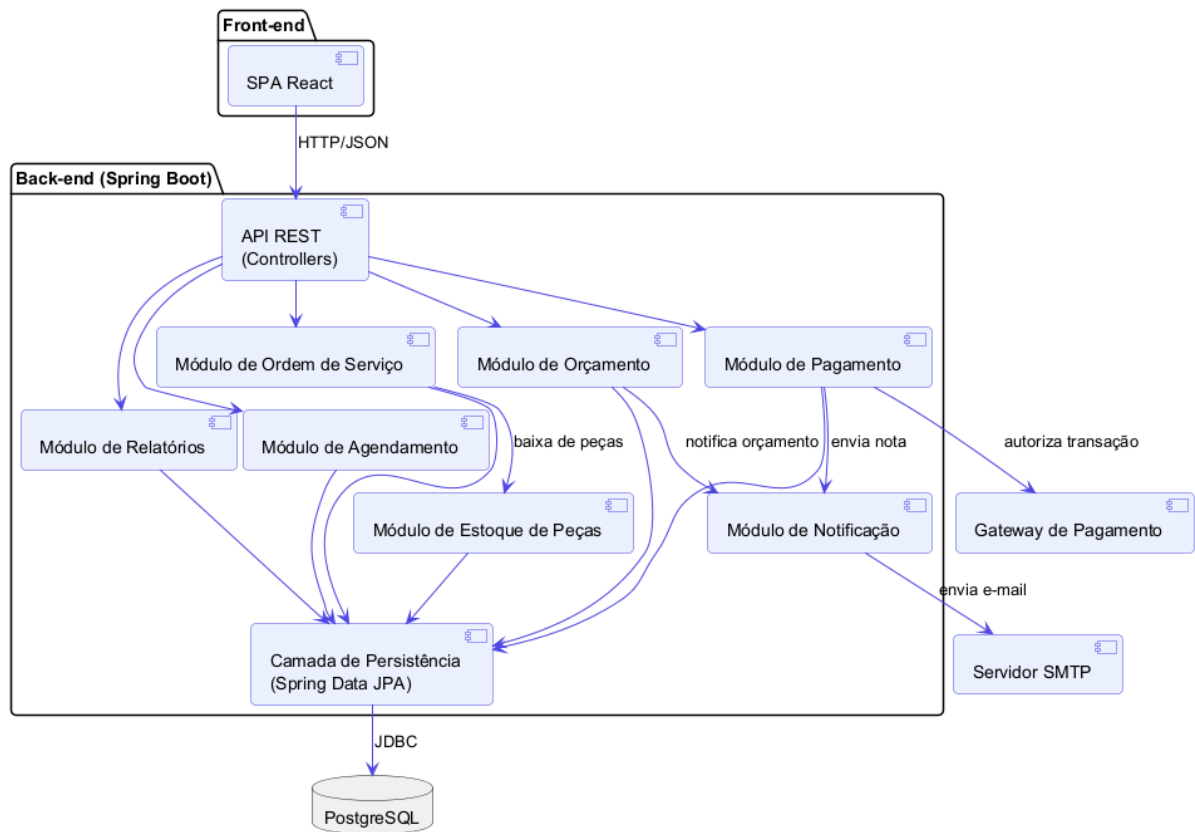


Figura 6 — Diagrama de Componentes do AutoFix.

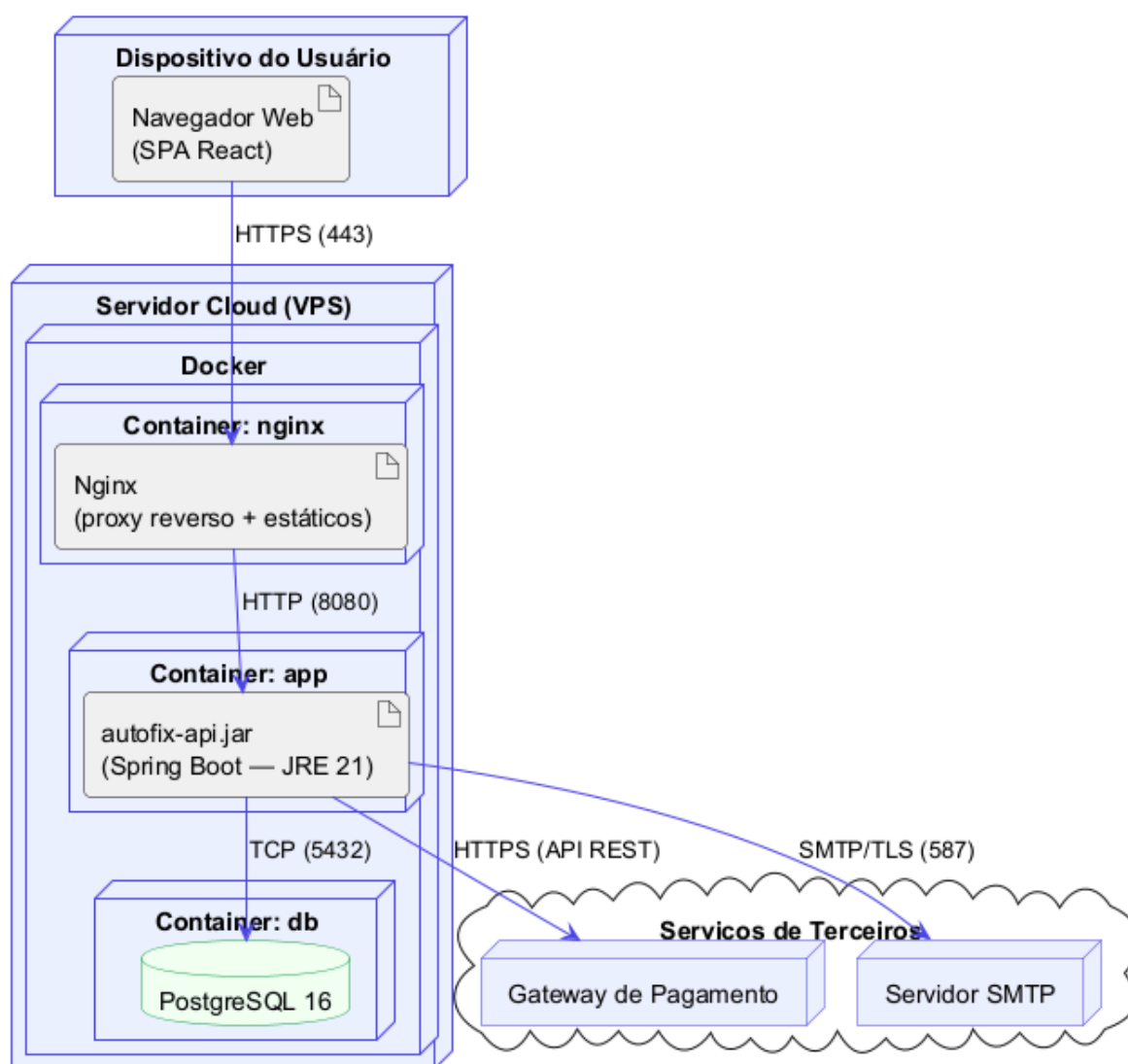
Diagrama de Implantação — AutoFix

Figura 7 — Diagrama de Implantação do AutoFix.

3.3. Diagrama de Classes

Diagrama de classes do sistema. As classes centrais do domínio são *OrdemServico* (com seus itens de serviço e de peça), *Orcamento*, *Agendamento* e *Pagamento*. Utiliza-se herança para *Pessoa* → *Cliente/Funcionario* e *Funcionario* → *Atendente/Mecanico*, além de enumerações para os estados e formas de pagamento.

Diagrama de Classes — AutoFix

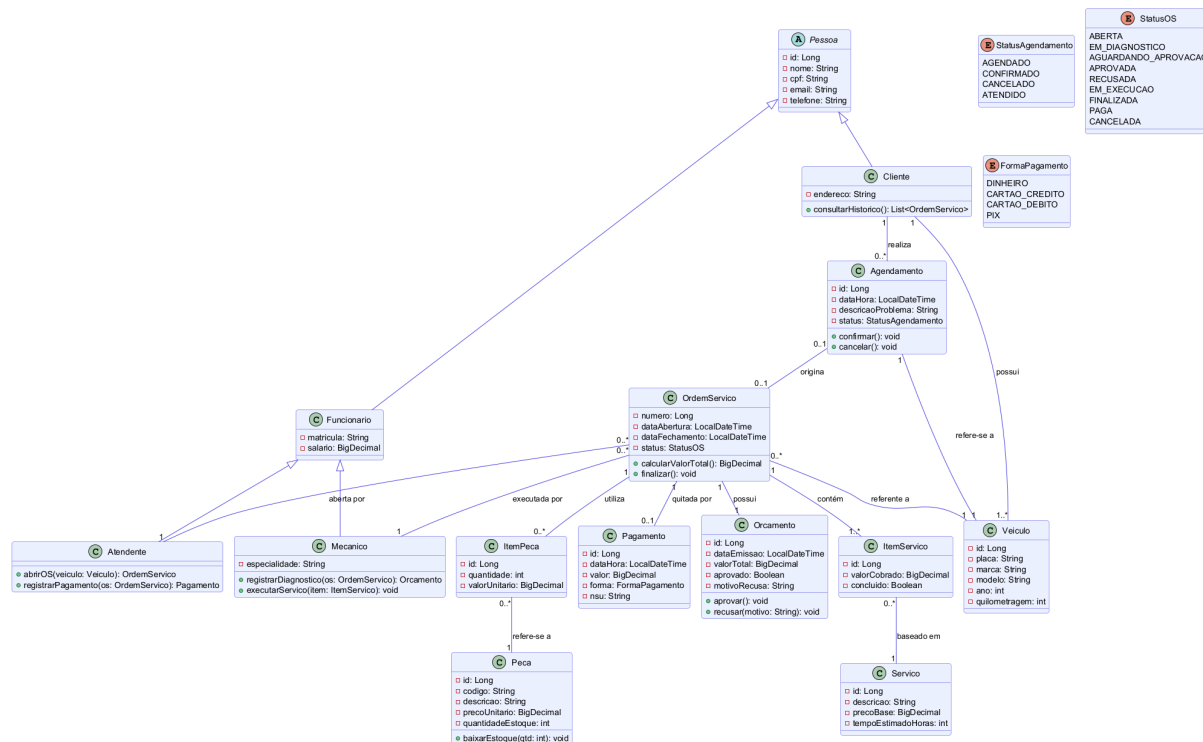


Figura 8 — Diagrama de Classes do AutoFix.

3.4. Diagramas de Sequência

Diagramas de sequência para realização de casos de uso. São detalhadas as realizações de **UC-05 — Aprovar Orçamento** e **UC-08 — Finalizar OS e Receber Pagamento**, mostrando a colaboração entre controladores, serviços, repositórios e sistemas externos.

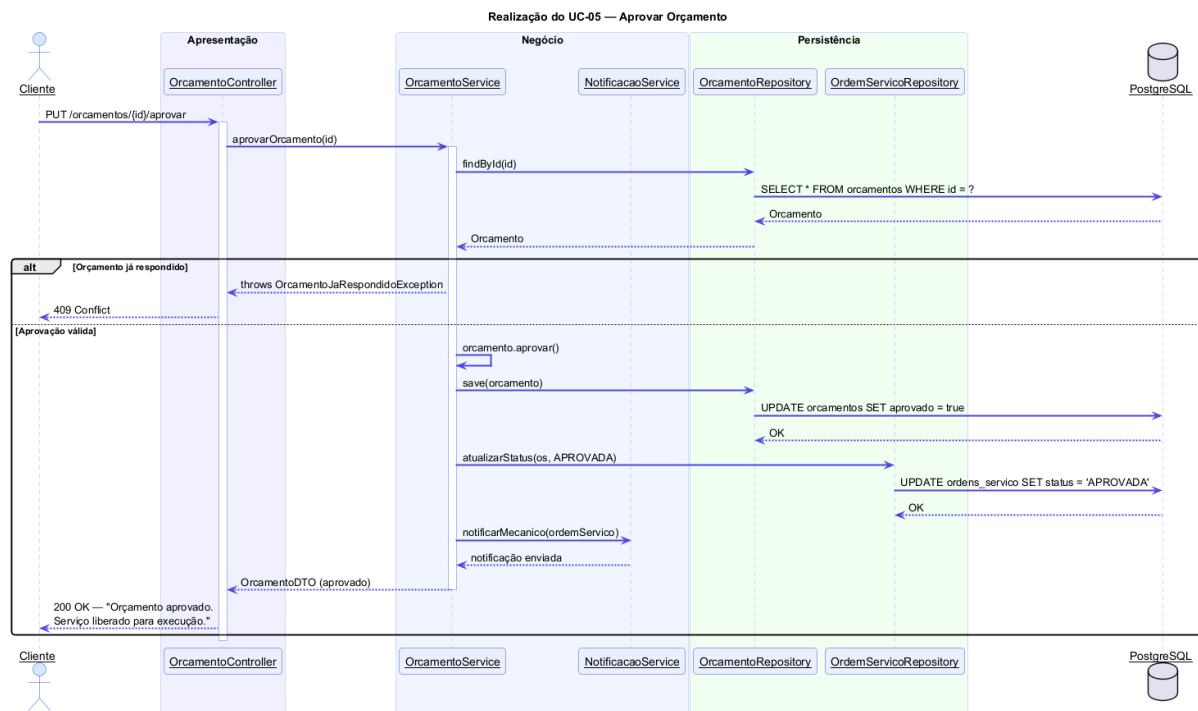


Figura 9 — Diagrama de Sequência da realização do UC-05.

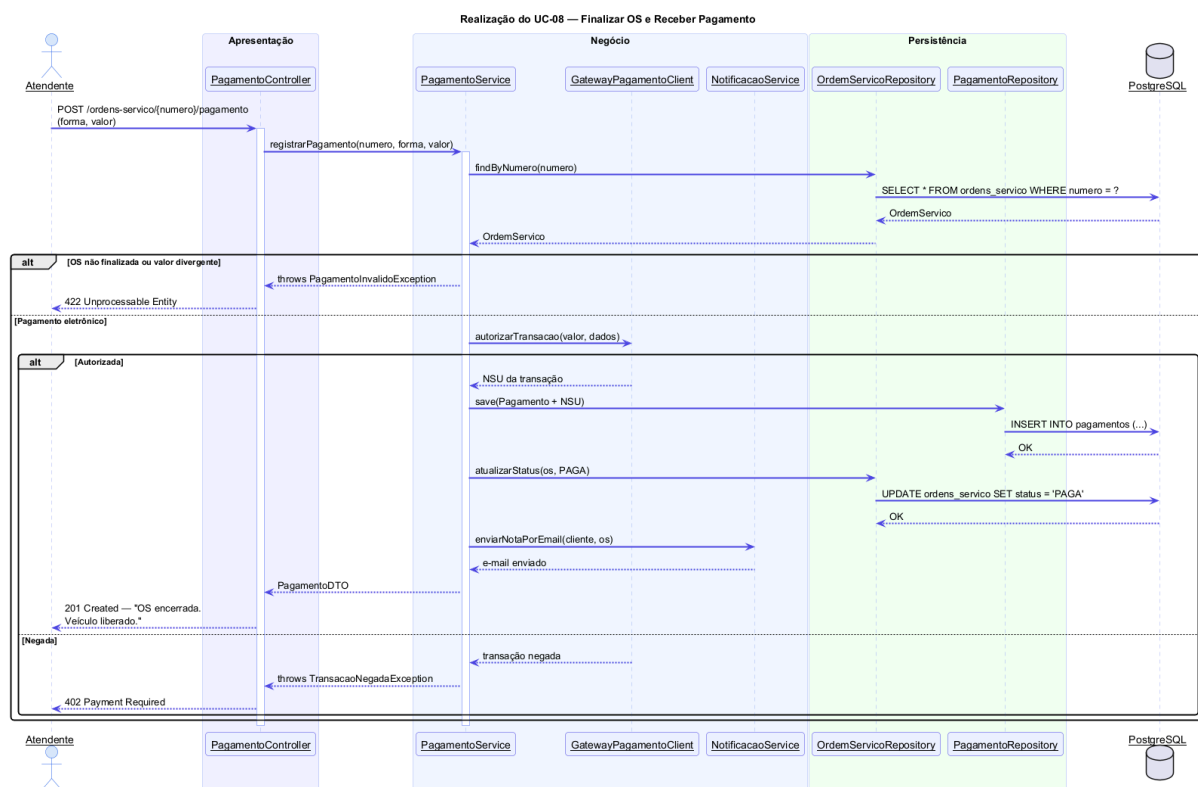


Figura 10 — Diagrama de Sequência da realização do UC-08.

3.5. Diagramas de Comunicação

Diagramas de comunicação para realização de casos de uso. As mesmas colaborações são apresentadas sob a perspectiva estrutural, com a numeração das mensagens indicando a ordem das interações.

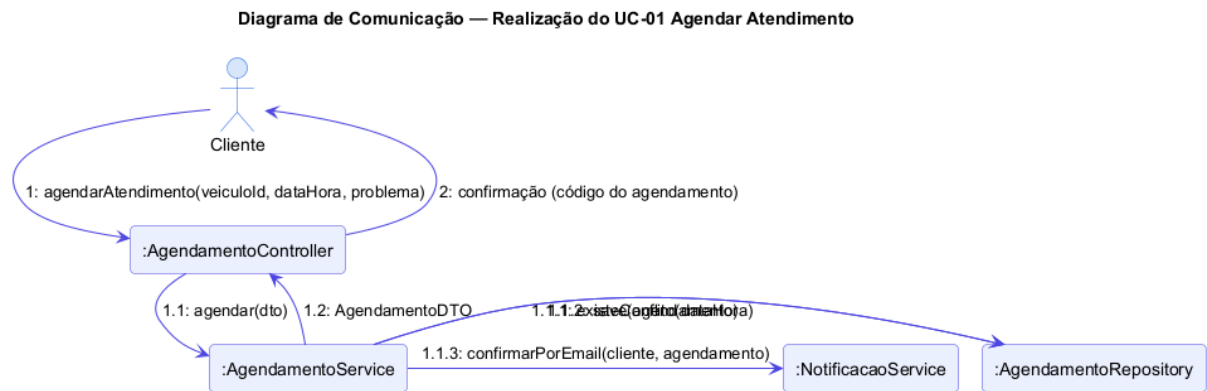


Figura 11 — Diagrama de Comunicação da realização do UC-01.

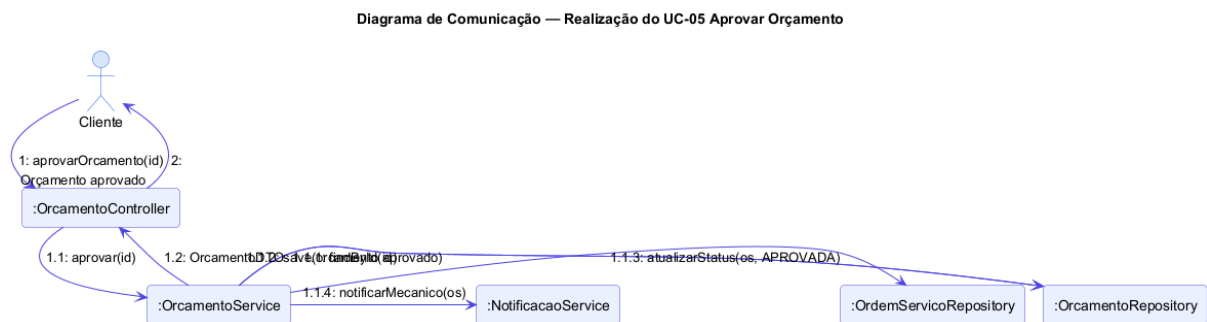
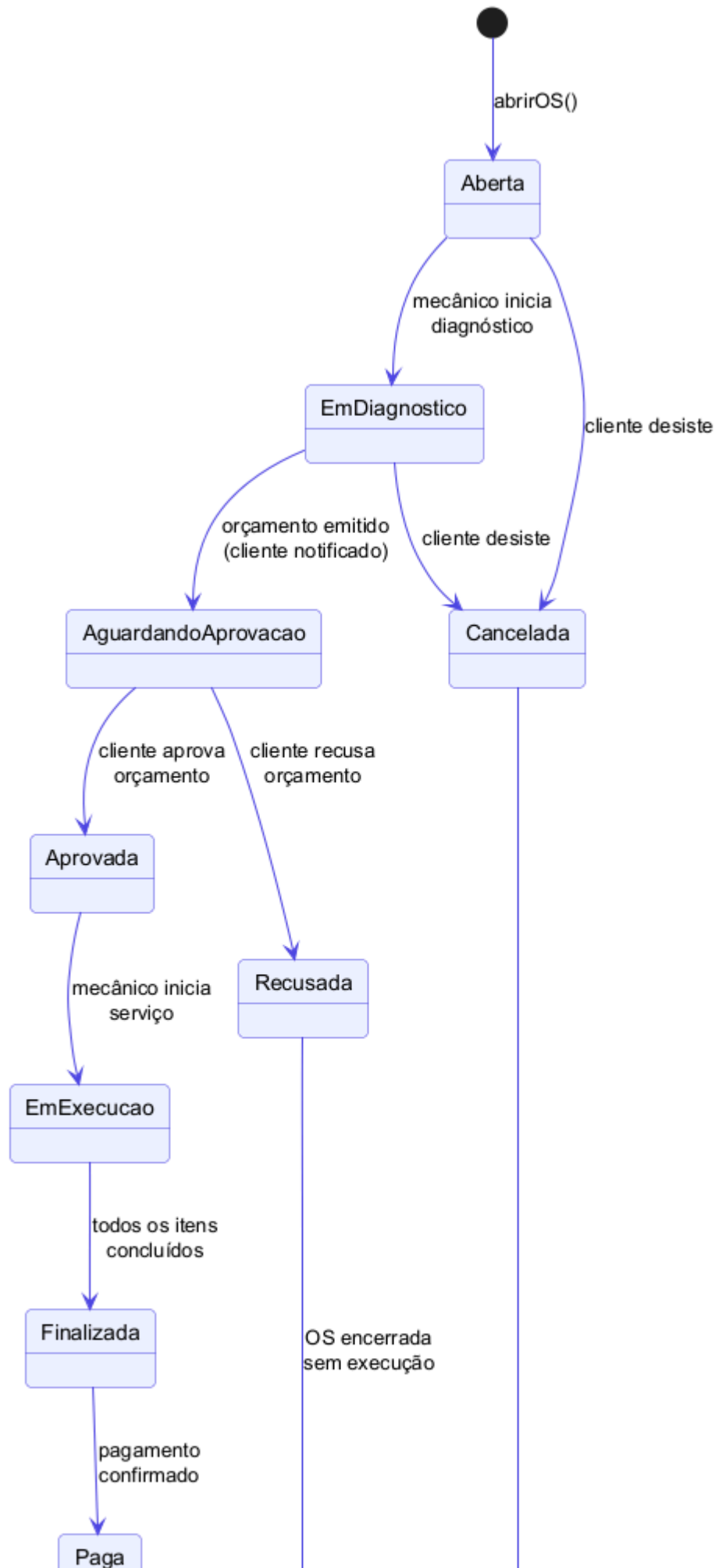


Figura 12 — Diagrama de Comunicação da realização do UC-05.

3.6. Diagramas de Estados

Diagramas de estados do sistema. O ciclo de vida da *Ordem de Serviço* é o mais rico do domínio: da abertura ao diagnóstico, passando pela aprovação (ou recusa) do orçamento, execução, finalização e pagamento — com possibilidade de cancelamento nas fases iniciais.

Diagrama de Estados — Ordem de Serviço (AutoFix)

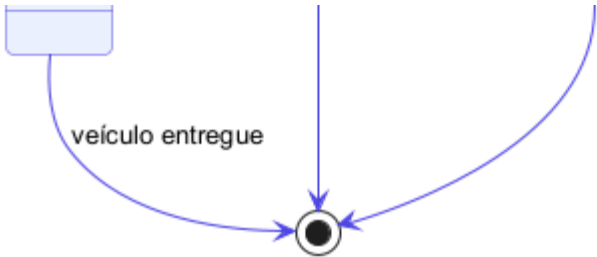


Figura 13 — Diagrama de Estados da Ordem de Serviço.

4. Modelos de Dados

Apresentam-se a seguir os esquemas de banco de dados e as estratégias de mapeamento entre as representações de objetos e não-objetos.

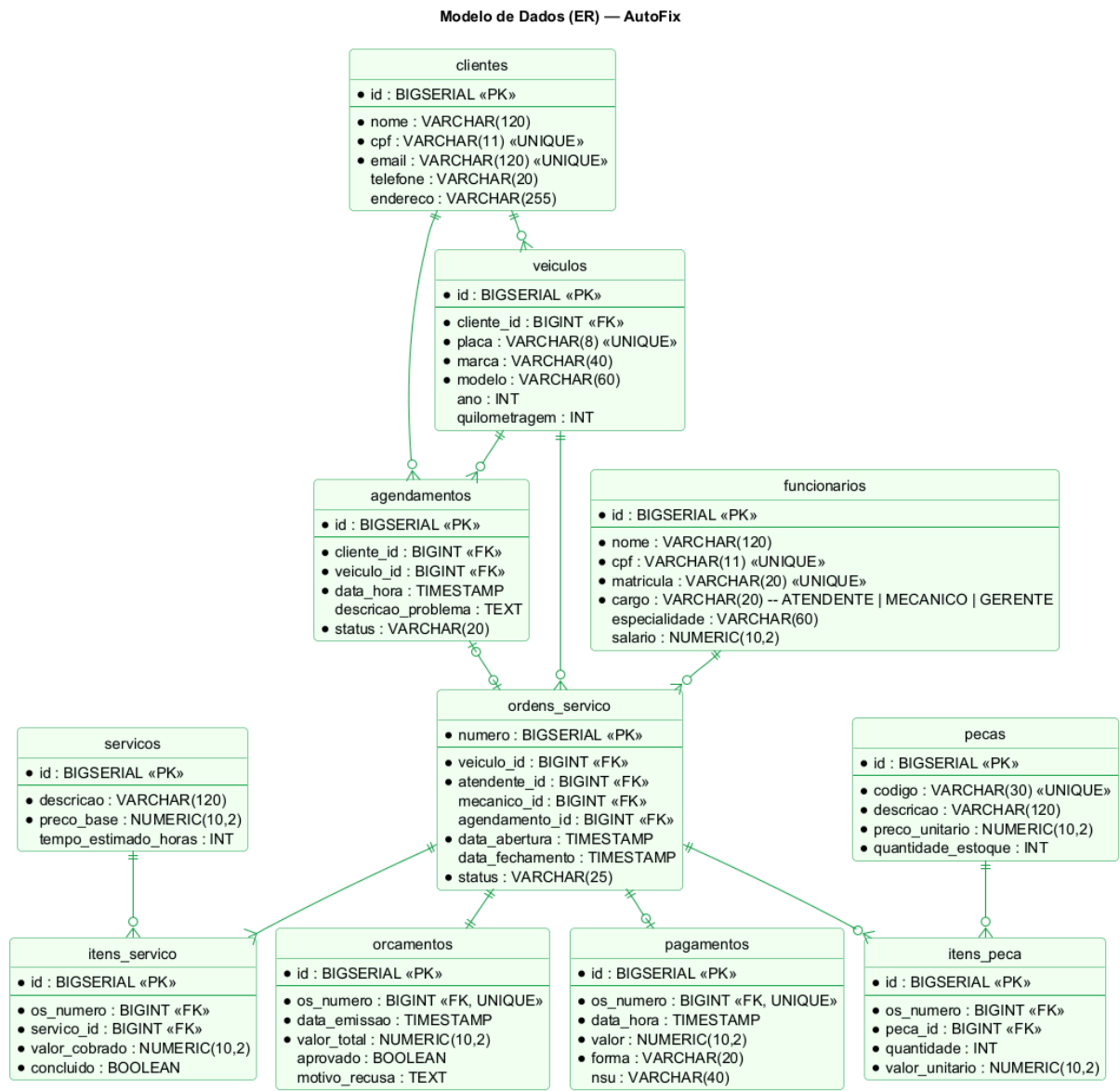


Figura 14 — Modelo de Dados (ER) do AutoFix.

Estratégia de mapeamento objeto-relacional (ORM): o mapeamento entre objetos e tabelas é realizado com **JPA/Hibernate**, segundo as decisões abaixo:

Decisão de mapeamento	Estratégia adotada
Herança <i>Pessoa</i> → <i>Cliente/Funcionario</i>	Uma tabela por classe concreta (TABLE_PER_CLASS achatada em <i>clientes</i> e <i>funcionarios</i>), pois os subtipos têm atributos e relacionamentos distintos e não há consultas polimórficas relevantes.
Herança <i>Atendente/Mecanico</i>	Discriminador <i>cargo</i> na tabela única <i>funcionarios</i> (SINGLE_TABLE), pois os campos específicos são poucos.
Enumerações (<i>StatusOS</i> , <i>FormaPagamento</i> , ...)	Persistidas como <i>VARCHAR</i> (EnumType.STRING), privilegiando legibilidade do esquema.
Associações 1:N (ex.: OS → itens)	Chave estrangeira no lado N, com carregamento LAZY e cascata na agregação composta (itens pertencem à OS).
Associações 1:1 (OS → Orçamento, OS → Pagamento)	Chave estrangeira com restrição UNIQUE na tabela dependente.
Valores monetários	<i>NUMERIC(10,2)</i> mapeado para <i>BigDecimal</i> , evitando erros de ponto flutuante.